CHAPTER-1

INTRODUCTION

Arduino and MQ-3 Sensor-based Alcohol Detector, a sophisticated system for detecting the presence of alcohol vapor in the environment around it. The project makes use of a MQ-3 gas sensor, which is particularly sensitive to alcohol vapors, and an Arduino microcontroller for processing the data. The combination of these components allows for the development of an efficient and portable alcohol detection system, resulting in a useful tool for applications such as breathalyzer devices and alcohol content monitoring in a variety of scenarios.. In order to ensure public safety, alcohol detectors are essential, particularly when it comes to averting drunk driving-related tragedies. We'll walk you through the steps of building an alcohol detector with an Arduino and a MQ-3 sensor in this blog article. This project can assist you in comprehending the fundamentals of alcohol detection and be a useful resource for keeping an eye on alcohol levels in a variety of settings.

The stuff we are going to used today to build an alcohol indicator is Arduino, LEDs, and an MQ-3 Alcohol sensor. There are many MQ-X sensors is available in the market for different uses but we are here going to use MQ-3 because it is best for detecting alcohol. The working of most of the MQ sensors is the same. They all contain a heating element that heats up a layer of conducting material whose resistance is continuously measured. Its resistance changes when fumes or smell from alcohol comes in contact with the MQ-3 sensor. **Arduino based alcohol detector** having the mq3 sensor the specification given above. which can be monitor in our iot based alcohol detector project.

The combination of these components allows for the development of an efficient and portable alcohol detection system, resulting in a useful tool for applications such as breathalyzer devices and alcohol content monitoring in a variety of scenarios.. In order to ensure public safety, alcohol detectors are essential, particularly when it comes to averting drunk driving-related tragedies This project can assist you in comprehending the fundamentals of alcohol detection and be a useful resource for keeping an eye on alcohol levels in a variety of settings

Executive Summary

. The Alcohol Detection System Using IoT (Internet of Things) is an advanced technology-driven solution designed to enhance safety and promote responsible behavior in environments where alcohol consumption is a concern. By leveraging IoT-enabled devices, this system provides real-time monitoring of alcohol levels, ensuring proactive measures are taken to mitigate risks associated with alcohol impairment. It integrates high-precision alcohol sensors with cloud-based data processing to deliver an efficient, scalable, and reliable detection solution.

This system works by using alcohol sensors that detect the concentration of alcohol in a person's breath. Once the alcohol level is detected, the data is transmitted via IoT devices to a cloud-based platform for real-time monitoring and analysis. The platform then triggers automated alerts, notifications, or actions based on predefined alcohol concentration thresholds, ensuring timely intervention. The solution can be integrated into a wide range of applications, such as vehicles, workplaces, and public spaces, where safety is a priority.

This IoT-based alcohol detection system is particularly beneficial in industries where safety is critical, such as transportation, healthcare, manufacturing, and hospitality. By detecting alcohol impairment in real-time, it helps prevent accidents, ensures a safe working environment, and promotes accountability among employees, drivers, and the public.

Additionally, the system's scalability and flexibility make it suitable for use in various scenarios, including personal vehicles, employee workstations, and public venues, where real-time data analytics and automated responses are essential. By offering a cost-effective and efficient way to monitor alcohol consumption, the system significantly enhances safety while reducing the potential for human error or negligence.

In conclusion, the Alcohol Detection System Using IoT is an innovative solution that combines IoT technologies with advanced sensor capabilities to address alcohol-related risks, providing a seamless and effective approach to alcohol monitoring and ensuring a safer environment across diverse settings.

1.1 SYSTEM SPECIFICATION

1.1.1 HARDWARE CONFICURATION

- MQ-3 Alcohol Sensor
- Arduino Uno
- 16x2 LCD Display
- I2c module
- 10k Variable Resistor
- 100 Ohm & 4.7k Ohm Resistors
- Green & Red LEDs
- Buzzer
- Jumper Wires
- Hard Jumper Wires
- 9V Battery & Battery Clip

HARDWARE DESCRIPTION

Fig 1.1 Arduino uno

The **Arduino Uno** is one of the most popular microcontroller boards in the Arduino family, widely used in various electronics and IoT projects due to its simplicity, flexibility, and ease of use. It is a versatile platform ideal for both beginners and experienced makers looking to develop various embedded systems and hardware applications.

The **Arduino Uno** is an open-source microcontroller board based on the **ATmega328P** chip, designed to be easy to use for both beginners and experienced developers. It features 14 digital input/output pins, 6 analog input pins, and various other components such as a USB connection for programming, a power jack, and a reset button. Operating at 5V with a 16 MHz clock speed, the board is programmable through the **Arduino IDE**.

allowing users to write code in a simple, C/C++-based language. The Uno is ideal for a wide range of projects, from basic electronics and IoT applications to robotics and home automation systems, thanks to its versatility, extensive library support, and large online community. While its limited processing power and memory may not suit very complex tasks, it remains one of the most popular and accessible platforms for prototyping and learning about embedded systems.

Fig 1.2 MQ-3 sensor

The **MQ-3** sensor is a popular and versatile gas sensor primarily used for detecting alcohol (ethanol) concentrations in the air. It operates on the principle of heating a sensing material and measuring the resistance change when exposed to various gases. The MQ-3 sensor is highly sensitive to alcohol vapors, making it ideal for applications like breath analyzers and alcohol detection systems.

It features both analog and digital outputs, allowing it to be easily interfaced with microcontrollers such as Arduino. The sensor has a wide detection range, typically between 0.04 to 4 mg/L, with a fast response time and the ability to detect low concentrations of alcohol. It requires a heating element to activate the sensor, which generally works at a voltage of 5V. The MQ-3 is often used in automotive breathalyzer systems, safety devices, and IoT-based alcohol detection systems, providing a simple and cost-effective solution for monitoring alcohol levels. However, it also responds to other gases such as smoke, methane, and carbon monoxide, so calibration and correct usage are essential for reliable alcohol detection.

- LCDs used for operation based on the principle of color emission by liquid crystals. And to display the message
- It is used to pull-up or pull-down resistors to ensure that the voltage level of a signal is maintained even when the signal is not being actively driven.
- It is used to connected or disconnected to change the settings or configuration of a particular component.
- 9-volt batteries are the preferred choice for plenty of handheld devices, including: Instrumentation and test devices. Wearable devices
- These LED lights are used for indication of alcohol detection.

1.1.2 SOFTWARE SPECIFICATION

Operating system : Windows

Programming language : embedded C

Application : Arduino IDE

SOFTWARE DESCRIPTION

ARDUINO IDE:

The **Arduino IDE** (Integrated Development Environment) is an open-source software platform used to write, compile, and upload code to Arduino boards. It is designed to be user-friendly, making it accessible to both beginners and experienced developers. The IDE supports the Arduino programming language, which is based on C/C++ but simplified for ease of use.

The interface of the Arduino IDE includes a text editor for writing code (known as "sketches"), a compiler to convert the code into machine language, and a serial monitor to communicate with the Arduino board. The IDE is compatible with a wide range of Arduino boards, such as the Arduino Uno, Mega, Nano, and others, and it allows users to easily upload code to the microcontroller via USB or serial connections.

Code Editor:

A simple text editor with syntax highlighting to write and edit Arduino programs.

Libraries:

It supports a wide array of pre-built libraries that simplify the integration of sensors, actuators, and communication modules.

Serial Monitor:

A tool for debugging, allowing the user to receive real-time data from the board or send commands to it.

Cross-platform:

The IDE is available for Windows, macOS, and Linux, making it highly versatile for different environments

1.Open Ardunio IDE as shown below

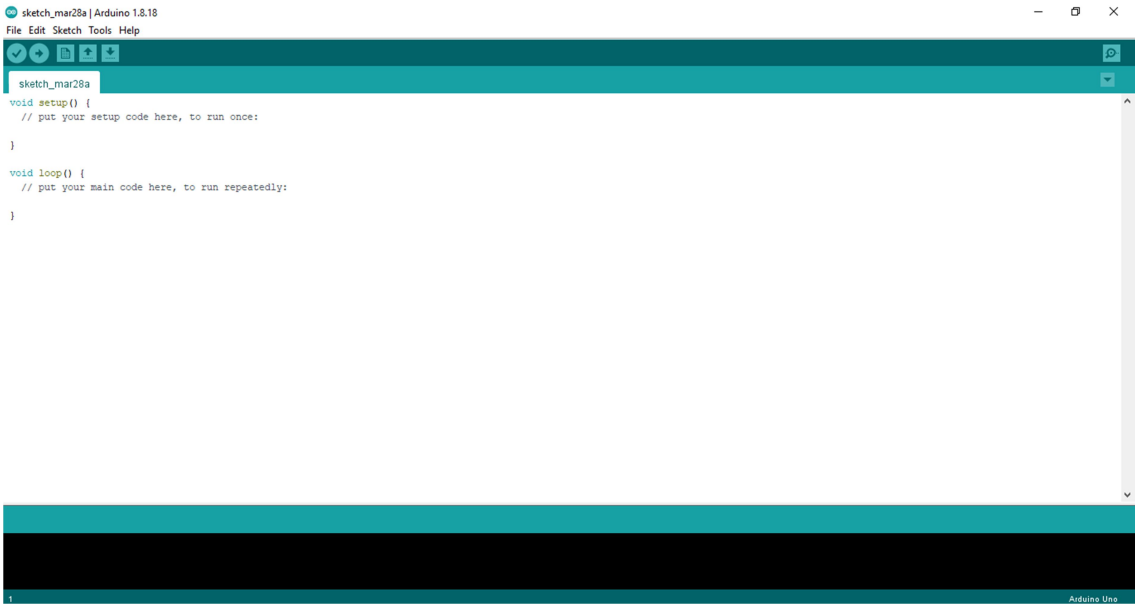


Fig 1.3 Arduino IDE

2.Select the COM port

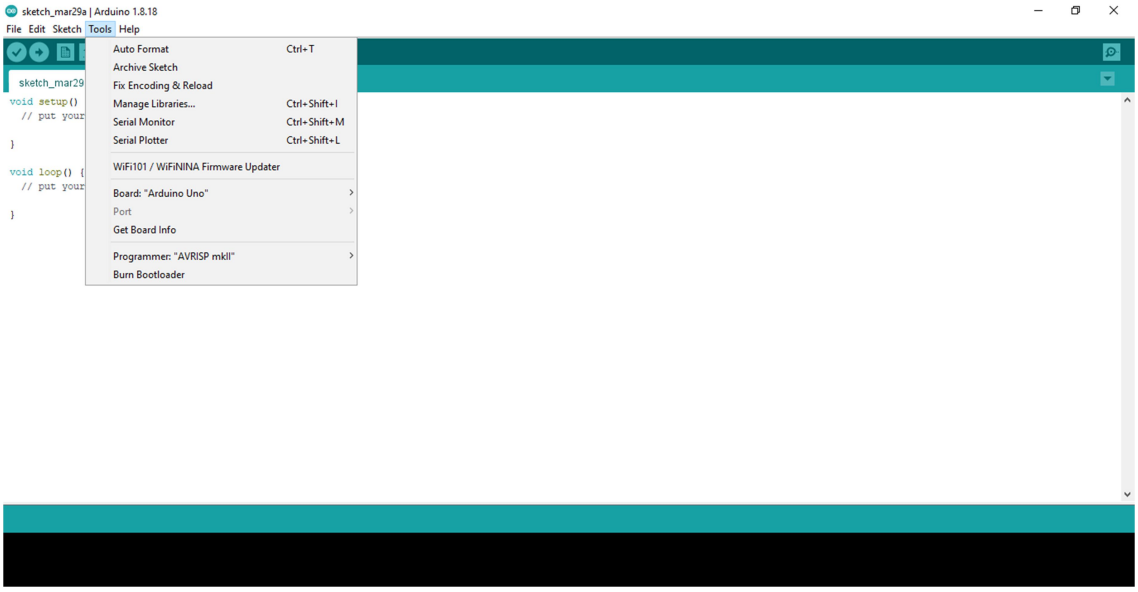


Fig 1.4 COM port to recognize the arduino

3.Select the Required Arduino as Shown below

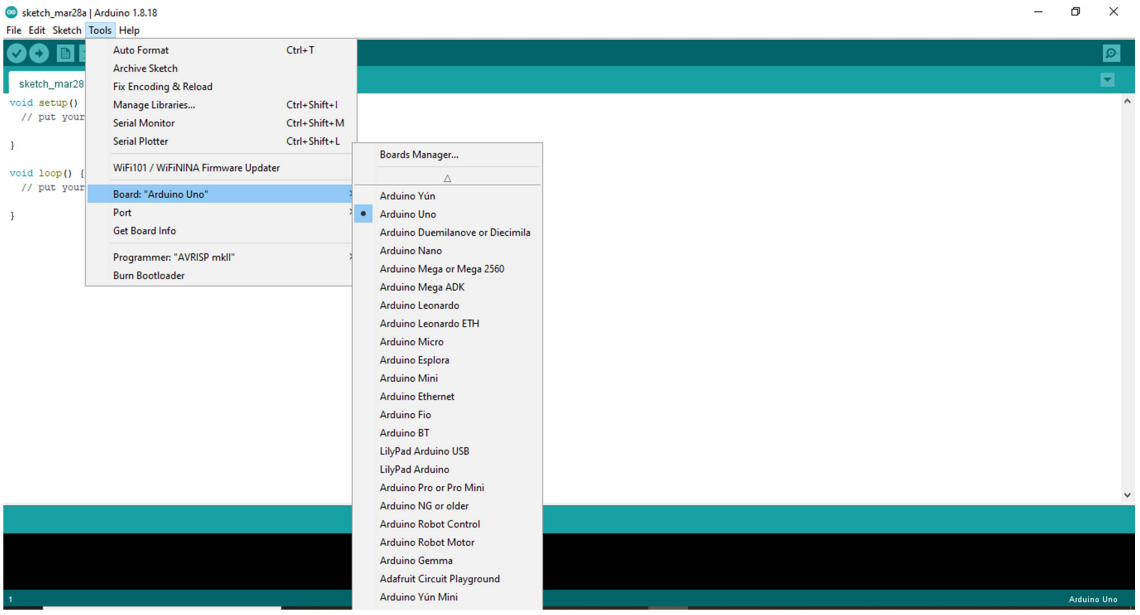


Fig 1.5 select the board which is to be connected.

4.Write the sketch in Arduino IDE as shown below

Fig 1.6 Write the sketch in Arduino IDE

5.Compile and upload the sketch in the Arduino as shown below

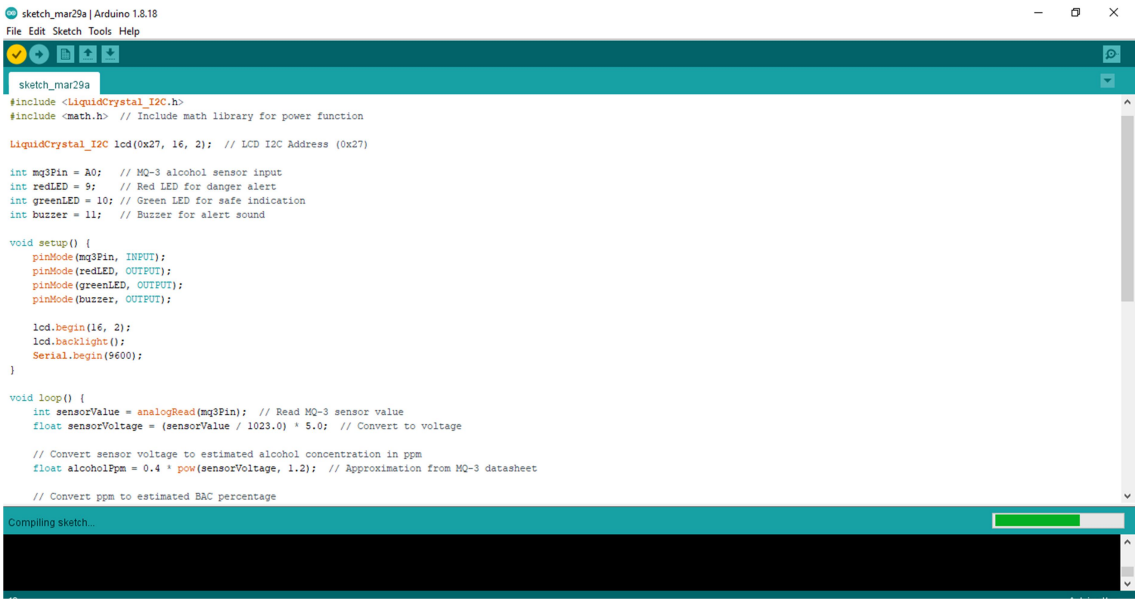


Fig 1.7 compile and upload the sketch in Arduino Uno

EMBEDDED C

Embedded C and are programming languages specifically tailored for developing applications in embedded systems, where low-level hardware interaction, real-time performance, and efficient use of resources are crucial. These languages allow developers to write software that directly interacts with the hardware while providing more control over system resources like memory and processing power.

CHAPTER-2

SYSTEM STUDY

2.1 EXISTING SYSTEM

2.1.1 DESCRIPTION OF EXISTING SYSTEM

The existing alcohol detection systems are commonly used in safety-critical environments like vehicles, workplaces, and public places to monitor and prevent the operation of machinery, vehicles, or equipment under the influence of alcohol. These systems are typically built using sensors to detect the presence of alcohol vapors in the air (such as a breathalyzer system), with a mechanism to alert the user or take preventive action. Most existing systems can be divided in the

2.1.2 FEAUTURES OF EXISTING SYSTEM

Standalone Alcohol Detection Systems (Breathalyzers):

- ✓ These systems use **gas sensors** (such as the **MQ-3** sensor or **Figaro TGS series**) to detect ethanol concentrations in the air.
- ✓ The user blows air into the device, and the sensor detects alcohol vapor. Based on the concentration of ethanol, the system either gives an alert or takes action like disabling the vehicle's ignition or preventing access to sensitive areas.
- ✓ The system's output is often shown on a **LED display** or sent as a **digital readout** to indicate the level of alcohol detected.
- ✓ **Advantages**: Simple to use, low cost, fast response time, and reliable for personal use in environments like bars or driving.
- ✓ **Limitations**: Limited data logging capabilities, standalone operation without remote monitoring, and lack of advanced features like IoT connectivity.

Drawbacks of the Existing System:

1. **Sensor Calibration**: Alcohol sensors need periodic calibration to ensure accurate detection. Environmental factors such as temperature, humidity, and other gases can affect the sensor's accuracy.

2. **False Positives**: The sensor may detect other gases like carbon monoxide or methane, leading to false readings. This is especially problematic in industrial environments with diverse gases in the air.
3. **Limited Functionality**: Basic systems without IoT features may lack remote monitoring capabilities, making it difficult to track alcohol consumption over time or across multiple locations.
4. **High Cost for Advanced Systems**: IoT-based systems can be more expensive to implement due to the additional components required for communication and data processing.

2.2 PROPOSED SYSTEM

2.2.1 DESCRIPTION OF PROPOSED SYSTEM

The proposed **Alcohol Detection System** aims to provide a more advanced, efficient, and reliable solution for detecting alcohol levels in individuals, particularly in safety-critical environments like vehicles, workplaces, and public spaces. The system combines real-time alcohol detection, IoT integration, and cloud-based data analysis to create an intelligent and automated safety mechanism. The primary objectives of the proposed system are to improve accuracy, enhance real-time monitoring capabilities, and enable remote notifications to provide a more comprehensive alcohol detection solution.

1. **Real-Time Alcohol Detection**:
    - The core functionality of the system is based on an alcohol sensor, such as the **MQ-3**, which detects ethanol vapors in the air. The sensor continuously monitors the concentration of alcohol and provides real-time data to the microcontroller.
    - The microcontroller processes this data and compares it with a predefined threshold to determine whether the alcohol concentration exceeds the safe limit.
    - If the alcohol concentration surpasses the threshold, the system triggers alerts (e.g., visual and auditory signals such as LEDs and buzzers) to warn the user.

2.2.2 FEATURES OF PROPOSED SYSTEM

- Users can set alcohol detection thresholds according to specific requirements.
- Configurable alert systems (LEDs, buzzers, SMS, email) for various scenario
- In high-risk applications (e.g., vehicles or heavy machinery), the system can disable ignition or restrict access if alcohol levels exceed safe limits.
- Restricts access to specific areas or equipment when alcohol is detected, ensuring safety in workplaces or restricted zones.
- Data filtering algorithms to minimize false positives and noise.
- Designed to operate on minimal power, suitable for battery-powered or portable applications.
- Includes a backup power option (e.g., rechargeable batteries).
- Simple setup and easy-to-use interface for configuration and monitoring.
- Real-time feedback on alcohol levels for quick decision-making..
- Low maintenance costs due to automatic calibration and remote monitoring.

CHAPTER-3

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

The file design for an Alcohol Detection System through IoT focuses on organizing both hardware and software components in a way that ensures scalability, maintainability, and clarity. The project is divided into key directories: **hardware**, **software**, **docs**, and **tests**. The **hardware** directory contains schematics, PCB designs, and firmware for the microcontroller that interfaces with the alcohol sensor (e.g., MQ-3). The **software** directory is split into frontend and backend, where the frontend includes a web dashboard for real-time monitoring, and the backend handles sensor data collection, storage, and API integration. Additionally, cloud integration for data sync and notifications is placed under The **docs** directory provides architecture diagrams, system overviews, and setup instructions, while **tests** ensures the robustness of the system with unit and integration tests. This structured design allows for seamless development, easy debugging, and smooth updates to the IoT-based alcohol detection system.

Sensing requirements: The sensing requirements for an Alcohol Detection System through IoT focus on selecting the right sensor, ensuring data accuracy, and maintaining efficient real-time monitoring. The core sensor, typically an **MQ-3 alcohol sensor**, must be sensitive to ethanol vapor, providing accurate readings within the typical range of 0.02% to 0.08% BAC, ideal for breathalyzer applications.

The sensor should have a fast response time, usually within a few seconds, to enable timely detection and action. Power efficiency is crucial, especially in portable applications, meaning low-power sensors and microcontrollers (such as ESP32 or Arduino) are preferred to prolong battery life. Additionally, the sensor must be able to operate under varying environmental conditions, with compensation for temperature and humidity fluctuations, to avoid inaccurate readings. The system should also allow for calibration and periodic recalibration to maintain accuracy over time. Finally, the alcohol sensor should be able to detect alcohol at specific thresholds, triggering alerts or notifications when levels exceed preset limits.

Central control unit: The **Central Control Unit (CCU)** in an Alcohol Detection System through IoT acts as the brain of the system, responsible for processing sensor data, making decisions, and communicating with other components like cloud servers or alert systems. It coordinates the entire system's functionality and ensures real-time alcohol detection and response.

- ✓ The CCU also handles other tasks such as data communication, power management, and possibly running local algorithms for filtering and analyzing sensor data
- ✓ The CCU is connected to one or more **alcohol sensors**, like the **MQ-3**, through analog or digital inputs. The sensor detects the concentration of ethanol vapor in the air, and the microcontroller receives the sensor's output.
- ✓ The CCU reads these signals, performs basic calibration (if needed), and applies logic to determine whether the detected level indicates alcohol consumption.
- ✓ It may also implement **noise filtering** and **smoothing algorithms** to account for any fluctuations or inaccuracies in sensor data.
- ✓ The CCU can store data locally (on an SD card or internal memory) or transmit it to the cloud for further analysis, historical tracking, or generating reports.
- ✓ Since many alcohol detection systems are designed for mobile or remote use (e.g., in vehicles), the CCU must be power-efficient. It should be able to run on battery power, with low-energy consumption during idle periods.
- ✓ For extended deployment, the system might use energy-saving features like **sleep modes** when idle or intermittent data transmission to minimize power usage
- ✓ A user might interact with the CCU via **buttons** for manual calibration or to reset the system.

3.2 INPUT DESIGN

The **input design** for an Alcohol Detection System through IoT focuses on capturing relevant data from external sources (such as alcohol sensors, user interactions, and environmental variables) and ensuring that the system processes these inputs effectively for real-time monitoring and decision-making.

1. Alcohol Sensor Input (Primary Input)

- **Sensor Type**: The primary input to the system is the **alcohol sensor** (e.g., **MQ-3**), which detects ethanol vapor in the air. The sensor typically outputs an **analog signal** corresponding to the alcohol concentration.
- **Analog-to-Digital Conversion**: The analog signal from the sensor is fed into the **microcontroller (MCU)**, where it is converted to a digital value using an **ADC (Analog-to-Digital Converter)**. This value represents the level of alcohol concentration (usually in parts per million, ppm).

2. Threshold Values (Pre-Configured Input)

- **Threshold Setting**: The system requires pre-configured input for **alcohol concentration thresholds**, which indicate when alcohol levels are considered dangerous (e.g., exceeding the legal driving limit). These thresholds will be compared against the sensor's readings to trigger alerts or other actions.

3. Environmental Factors (Optional Inputs)

- **Temperature Sensor**: A sensor like the **DHT11/DHT22** can be used to measure the ambient temperature, allowing the system to adjust alcohol readings based on temperature variations.
- **Humidity Sensor**: Humidity can also impact sensor readings. A **DHT sensor** or similar can be used to measure relative humidity and fine-tune the alcohol detection logic.

4. Power Input

- **Power Supply**: The system requires an input for power, usually provided by a **battery**, **USB power**, or **external power supply**. This is particularly important in mobile or portable systems like those used in vehicles.

5. Communication Input (Optional for Real-Time Monitoring)

- **MQTT/Broker Input**: For cloud-based IoT systems, the input design must also include configuration details for **MQTT brokers** or other cloud communication methods.

3.3 OUTPUT DESIGN

3.3.1 LED ON/OFF

A local display can provide the user with real-time information about alcohol levels and system status. The display can show the following:

Alcohol Concentration:

The real-time percentage of ethanol in the air or an equivalent unit (e.g., **BAC** or **ppm**).

Status Messages:

Messages like "No Alcohol Detected," "Alcohol Detected," or "System Needed."

Warning Alerts:

Text such as “Warning: Alcohol Level High!” or “Safe to Drive” based on predefined thresholds.

Status Indicators:

Simple LEDs (e.g., **green** for safe, **yellow** for caution, **red** for danger) can be used to quickly indicate the alcohol level status.

3.3.2 Audio Output (Alarm/Alert)

A built-in **buzzer** or **siren** can be triggered when alcohol concentration exceeds a predefined threshold, alerting users to a critical situation.

High Alcohol Level:

If the detected alcohol level surpasses the safe threshold (e.g., the legal BAC for driving), the system can trigger an **alarm** to warn the user, or even lock the vehicle's ignition if used in that context.

Warning Sound:

Continuous sound can be used for lesser violations (e.g., cautionary levels) or during sensor calibration processes.

3.4 CODE DESIGN

The **Embedded C** code design for an Alcohol Detection System is centered around efficient handling of sensor inputs, real-time data processing, and system outputs, while ensuring minimal resource consumption, crucial for embedded systems with limited memory and processing power. The system typically interfaces with an alcohol sensor, such as the **MQ-3**, using analog-to-digital conversion to capture alcohol concentration levels. In Embedded C, the code is structured to initialize and configure hardware peripherals (like ADCs, timers, and I/O pins) and run continuously in a loop to monitor sensor readings and the **Embedded C** is ideal for building responsive, reliable alcohol detection systems in resource-constrained IoT applications**.**

The structure of sketches

A blank arduino sketches has two functions:

- Setup()
- Loop()

3.5 SYSTEM DEVELOPMENT

3.5.1 DESCRIPTION OF MODULES

1. Design phase

The **Design Phase** is a critical stage in the development of the Alcohol Detection System through IoT, as it sets the foundation for how the system will function, how components will interact, and how the final product will meet user requirements. The design phase typically involves

SYSTEM ARCHITECTURE

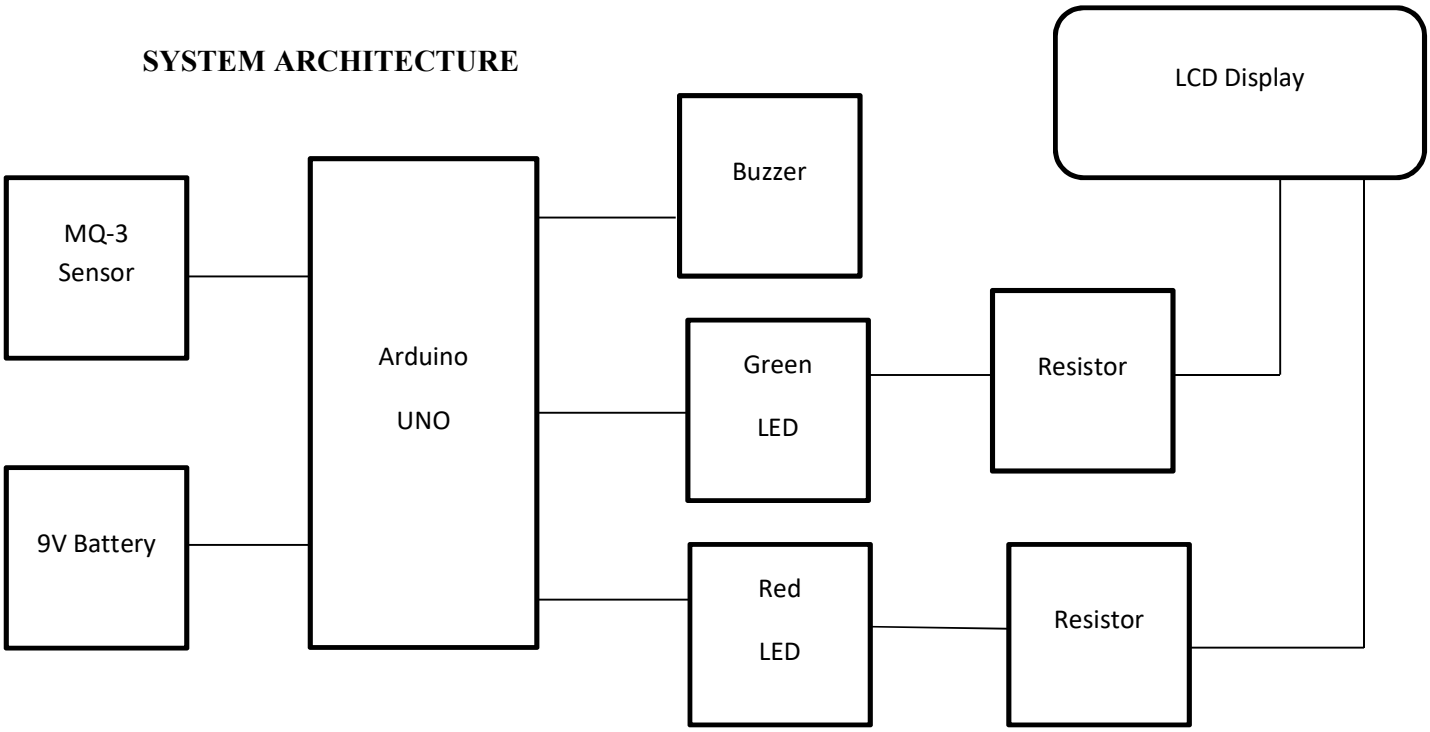


Fig 3.1 System Architecture.

2. Implementation phase:

The **Implementation Phase** is where the design and planning laid out in the previous phases are brought to life. This is the stage where the actual development work takes place, and the system begins to take form. During this phase, the hardware components are assembled, the software is written and integrated, and the system is tested for functionality and performance.

MODULES:

- Alcohol Sensing Module

Objective: To detect the alcohol concentration in the air, typically using a gas sensor.

Components:

- **Alcohol Sensor (MQ-3 or MQ-7)**: This sensor detects alcohol vapors in the surrounding environment. It provides an analog output, which varies with the concentration of alcohol in the air.
- **Signal Conditioning Circuit**: To convert the analog signal into a digital format that can be processed by the microcontroller, an analog-to-digital converter (ADC) is typically used.

Working:

- The **MQ-3 sensor** detects alcohol vapors and sends an analog signal to the microcontroller.
- The concentration of alcohol in the air will directly affect the resistance of the sensor's heating element.
- The microcontroller reads the analog value and processes it to determine the alcohol concentration.

- Microcontroller (Central Processing Unit)

Objective: To process sensor data, perform threshold comparisons, control output devices, and manage communication with cloud servers.

Components:

- **Arduino**: These microcontrollers are used to read sensor data, execute control logic, and interface with external devices like LEDs, buzzers, and communication modules.

Working:

- **Sensor Data Processing**: The microcontroller reads the sensor's analog or digital output. It converts the data into a meaningful value (e.g., percentage of alcohol concentration).
- **Threshold Logic**: The microcontroller compares the alcohol concentration to a predefined threshold. If the concentration exceeds the threshold, the system triggers an output (e.g., alert or control action).
- **Control Logic**: If the alcohol concentration is unsafe, the microcontroller will activate the buzzer, red LED, or relay (for controlling external systems,).

❖ Output Control Module

Objective: To provide feedback or trigger actions based on alcohol concentration levels.

Components:

- **LEDs**: **Green LED** (safe) and **Red LED** (unsafe).
- **Buzzer**: An audible alarm to alert when unsafe alcohol levels are detected.

Working:

- **LEDs**: If alcohol concentration is within safe limits, the green LED is turned on. If the concentration exceeds the safe threshold, the red LED is activated.
- **Buzzer**: If the alcohol concentration is above the threshold, the buzzer sounds to alert the user.

❖ **Power Supply Module**

Objective: To provide a stable power source to all components in the alcohol detection system.

Components:

- **Power Supply**: Depending on the system's mobility, power can be supplied either via **USB**, **battery**, or **solar panels**.
- **Voltage Regulators**: To provide the correct voltage levels to the microcontroller, sensors, and communication module.

Working:

- The power supply ensures that all components are powered correctly. For example, the ESP32 might require 5V, while the alcohol sensor might require 3.3V.
- In some cases, rechargeable batteries or solar power can be used for remote applications.

❖ System Integration Module

Objective: To ensure that all modules (hardware and software) work cohesively together to achieve the desired functionality.

Components:

- **Integrated Software (Code)**: Code for reading sensor values, comparing them with thresholds, and triggering outputs.
- **Integration of All Modules**: All components must be wired correctly and tested to ensure that data flows seamlessly from one part of the system to another.

Working:

- The system is integrated to ensure the sensor detects alcohol, the microcontroller processes the data, output devices are activated when necessary, and data is sent to the cloud for monitoring.
- Debugging and testing are crucial in ensuring that the system is fully operational before deployment

.

3. Intergration phase:

The **Integration Phase** is a critical stage in the development of the Alcohol Detection System through IoT, as it focuses on combining individual modules and components into a fully functional system. This phase involves the final testing of the entire system to ensure that hardware, software, and communication protocols work together seamlessly.

4. Deployment phase:

The **Deployment Phase** is the final step in the development cycle of the Alcohol Detection System through IoT. This phase involves transitioning the system from development and testing to real-world implementation. The deployment phase ensures that the system is operational in its intended environment and that users can effectively utilize the system for its designed purpose.

5. Maintenance and support:

The **Maintenance and Support** phase is crucial to ensure the long-term performance, reliability, and effectiveness of the Alcohol Detection System after its deployment. This phase includes monitoring, troubleshooting, software updates, user support, and addressing any potential issues that may arise during the system's operational lifecycle. Proper maintenance ensures that the system continues to operate smoothly, provide accurate readings, and deliver timely alerts to the users.

CHAPTER-4

4. TESTING AND IMPLEMENTATION

The **Testing and Implementation** phase is a crucial step in the development lifecycle of the Alcohol Detection System through IoT. This phase ensures that the system works as intended under real-world conditions, meets all the required specifications, and is ready for deployment. It involves multiple stages of testing, debugging, and system validation to verify the performance, functionality, and reliability of the system before its official implementation.

1. Testing Phase

Key Objectives:

- To verify that the Alcohol Detection System functions as expected in different environments.
- To identify any system flaws, bugs, or issues that may impact its effectiveness.
- To ensure that the system performs consistently and accurately over time.

Types of Testing:

A. Unit Testing

- **Objective**: Test individual components (like the alcohol sensor, microcontroller, and communication modules) to ensure they work as expected.
- **Activities**:
    - Test the **alcohol sensor** (e.g., MQ-3 or MQ-7) to make sure it detects alcohol concentrations and provides accurate readings.
    - Test the **microcontroller** (e.g., Arduino or ESP32) to verify that it processes sensor data and triggers the correct outputs (e.g., LED, buzzer)..
    - Validate **power consumption** and ensure that the system operates within the required energy constraints.

B. Integration Testing

- **Objective**: Ensure that all components of the system work together as a cohesive whole.
- **Activities**:
  - Integrate the **alcohol sensor**, **microcontroller**, and **output devices** (e.g., LEDs, buzzers).
  - Verify that when alcohol levels exceed the threshold, the system triggers the correct alerts (e.g., visual, auditory alerts, and cloud notifications).

C. Functional Testing

- **Objective**: Verify that the system's functionality meets the desired specifications and use cases.
- **Activities**:
  - Test the system's ability to **accurately measure alcohol levels** in different concentrations and environmental conditions.
  - Ensure that the system **alerts users** when alcohol concentration exceeds the predefined threshold.
  - Validate that the **output devices (LEDs, buzzer)** work properly in both high and low alcohol concentration scenarios.

D. Performance Testing

- **Objective**: Assess how well the system performs under different conditions, including stress testing.

- **Activities**:

  - Test the system's **response time** to alcohol detection. The system should respond immediately when the alcohol level crosses the threshold.
  - Stress test the **system under continuous use** to ensure that the alcohol sensor and communication modules don't overheat or malfunction.
  - Measure the system's **latency** and ensure real-time data transmission from the sensor to the cloud platform.

- Test the system's **stability** under various network conditions (Wi-Fi or Bluetooth dropouts, slow internet speeds).

.

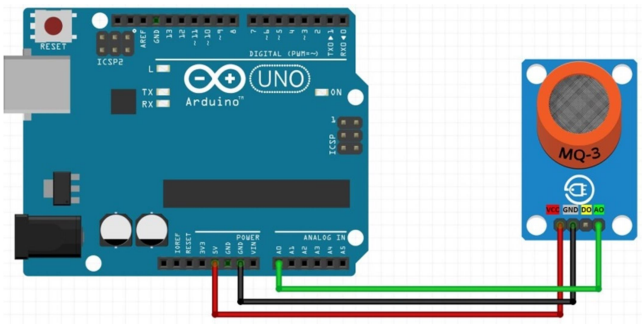


Fig 4.1 Arduino to mq-3 sensor connection

CHAPTER -5

CONCLUSION

The **Alcohol Detection System through IoT** is an innovative and crucial technology aimed at improving safety and reducing the risks associated with alcohol consumption in various environments. Throughout the development lifecycle, from design to deployment, key phases such as testing, integration, and implementation ensure the system's reliability, accuracy, and effectiveness in real-world scenarios.

By leveraging **IoT technologies**, this system provides real-time monitoring, remote data analysis, and automatic alerts, helping to prevent dangerous situations where individuals might be under the influence of alcohol, such as in vehicles or workplaces. The **sensor calibration**, **software updates**, and **continuous monitoring** ensure that the system remains effective over time, providing accurate readings and timely notifications.

The comprehensive testing phases, including **unit testing**, **functional testing**, and **performance evaluation**, confirm that the system can function under different environmental conditions, detect alcohol accurately, and deliver consistent results. The **implementation phase** further guarantees that the system is properly integrated into its intended environment, is easy to use, and complies with relevant standards.

Ultimately, the success of this system hinges on its ability to integrate seamlessly with existing technologies, provide actionable insights to users, and ensure continuous operation with minimal maintenance. The Alcohol Detection System through IoT not only enhances personal and public safety but also contributes to a broader goal of promoting responsible behavior and safeguarding individuals from the harmful effects of alcohol consumption. By delivering reliable, real-time data and alerts, the system makes a significant contribution to improving safety in various contexts, whether in vehicles, workplaces, or public spaces.

CHAPTER-6

BIBLIOGRAPHY

1. **Bhattacharya, P., & Gupta, S. (2020).***"IoT-Based Alcohol Detection and Monitoring System."*
   Available at: https://www.ijarcs.info
2. **Mishra, S., & Kumar, R. (2019).***"Embedded Systems Design for Alcohol Detection Using IoT."*.
3. **Zhou, H., et al. (2018).***"Cloud-Based IoT for Real-Time Alcohol Detection and Monitoring."*
4. **Kumar, S., & Soni, M. (2019).***"Smart Alcohol Detection System Using IoT: A Review."*
   Available at: https://www.ijatcse.com
5. **Karan, R., & Patel, S. (2020).***"IoT Based Smart Alcohol Detection System for Vehicles.*
   Available at: https://www.ijcaonline.org
6. **Ghimire, S., & Shrestha, R. (2018).***"Design and Development of IoT-Based Alcohol Detection System for Monitoring.*
   International Journal of Computer Science and Mobile Computing, 7(9), 89-98.
   Available at: https://www.ijcsmc.com.
7. **Arduino (2021).***"Getting Started with Alcohol Sensor MQ-3."*
   Arduino Documentation.
   Available at: https://www.arduino.cc/en/Guide
8. **Monish, M., & Nithya, A. (2018).***"Implementation of Alcohol Detection System Using IoT and Embedded C Programming."*
9. **Raza, U., et al. (2020).***"A Comprehensive Survey on IoT-Based Alcohol Detection Systems."*
10. **Techopedia (2022).***"What is Internet of Things (IoT)?"*
    Techopedia.
    Available at: https://www.techopedia.com

DATAFLOW DIAGRAM:

LEVEL-0 CONTEXT DIAGRAM

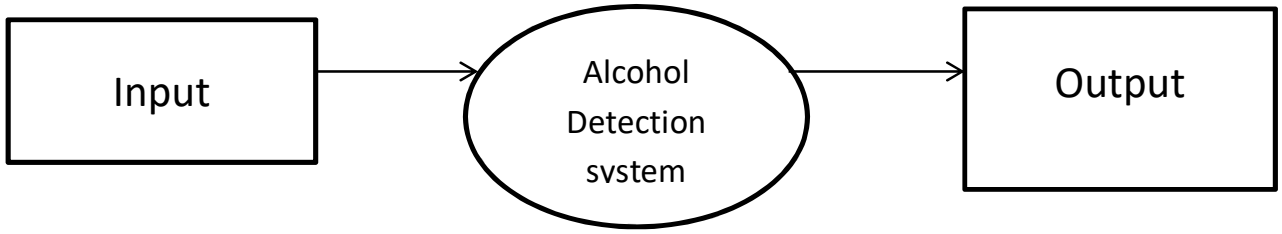


Fig 6.1 Level-0 Data flow Diagram

LEVEL-1 DATA FLOW DIAGRAM

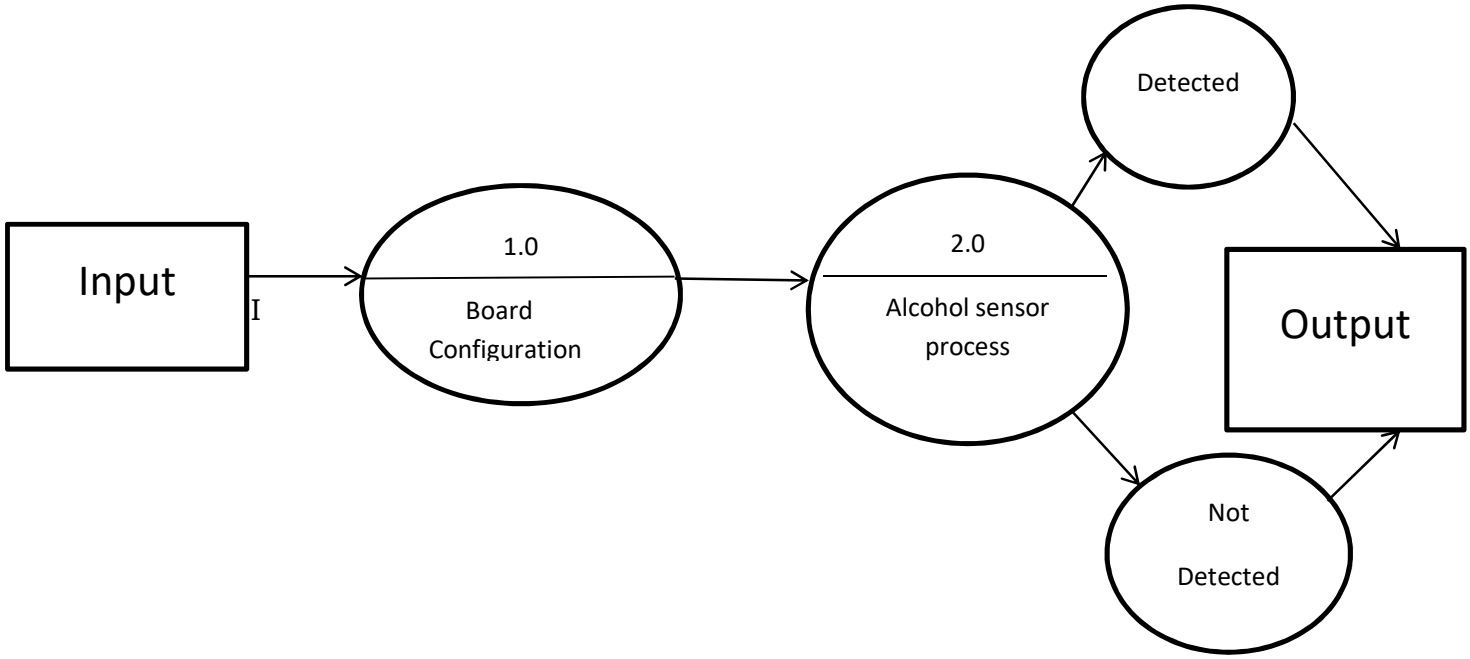


Fig 6.2 Level-1 Date Flow Diagram

SAMPLE CODING FOR LED

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <math.h>  // Include math library for power function

LiquidCrystal_I2C lcd(0x27, 16, 2);  // LCD I2C Address (0x27)

int mq3Pin = A0;   // MQ-3 alcohol sensor input

int redLED = 9;    // Red LED for danger alert

int greenLED = 10; // Green LED for safe indication

int buzzer = 11;   // Buzzer for alert sound

void setup()

 {

   pinMode(mq3Pin, INPUT);

   pinMode(redLED, OUTPUT);

   pinMode(greenLED, OUTPUT);

   pinMode(buzzer, OUTPUT);

   lcd.begin(16, 2);

   lcd.backlight();

   Serial.begin(9600);

}
```

```
void loop()
 {
   int sensorValue = analogRead(mq3Pin);  // Read MQ-3 sensor value
   float sensorVoltage = (sensorValue / 1023.0) * 5.0;  // Convert to voltage
   // Convert sensor voltage to estimated alcohol concentration in ppm
   float alcoholPpm = 0.4 * pow(sensorVoltage, 1.2);  // Approximation from MQ-3 datasheet
   // Convert ppm to estimated BAC percentage
   float alcoholPercentage = (alcoholPpm * 0.21) / 1000;
   // Display on LCD
   lcd.clear();
   lcd.setCursor(0, 0);
   lcd.print("Alcohol %: ");
   lcd.setCursor(0, 1);
   lcd.print(alcoholPercentage, 3);  // Show 3 decimal places
   lcd.print(" %");
   Serial.print("Alcohol %: ");
   Serial.println(alcoholPercentage, 3);
   // Set threshold for danger alert
   if (alcoholPercentage > 0.08) {  // 0.08% is the legal BAC limit in many countries
```

```
        digitalWrite(redLED, HIGH);

        digitalWrite(greenLED, LOW);

        digitalWrite(buzzer, HIGH);

        lcd.setCursor(10, 1);

        lcd.print(" DANGER!");

    }

Else

 {

        digitalWrite(redLED, LOW);

        digitalWrite(greenLED, HIGH);

        digitalWrite(buzzer, LOW);

    }

    delay(1000);  // Refresh every 1 second

}
```

SAMPLE INPUT

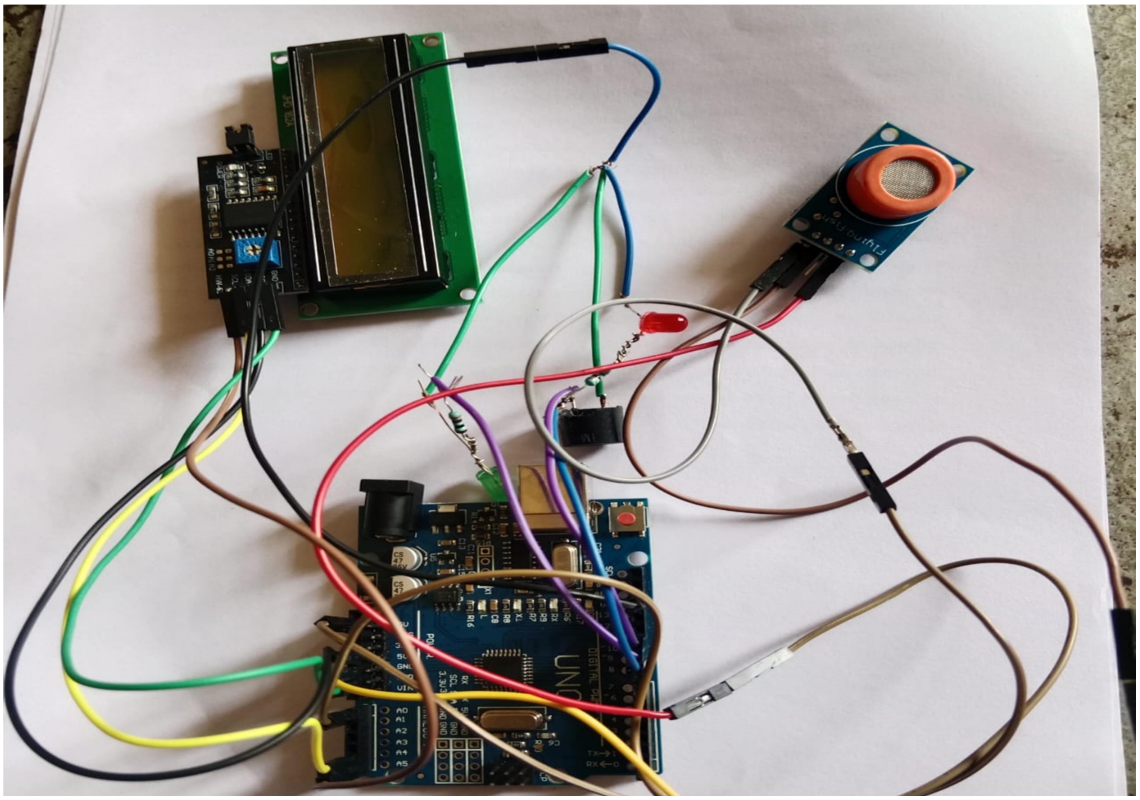
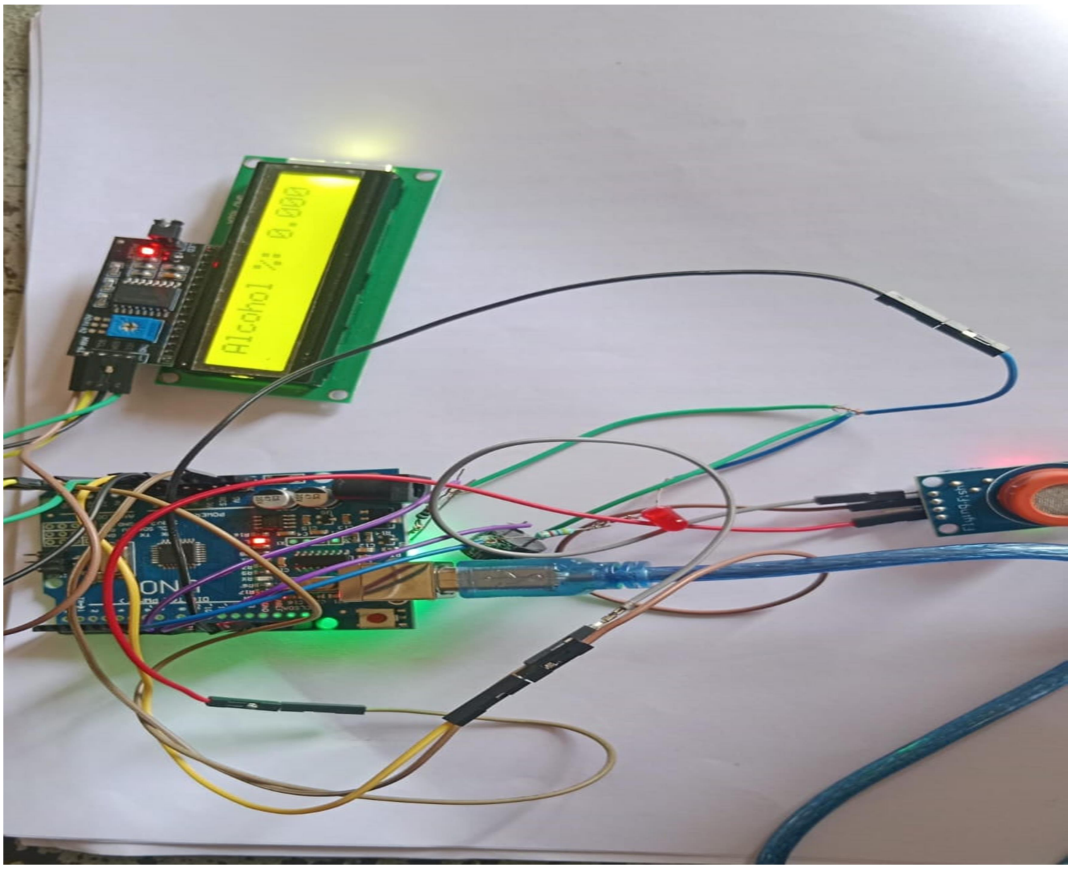


Fig 6.3 Sample Input

SAMPLE OUTPUT



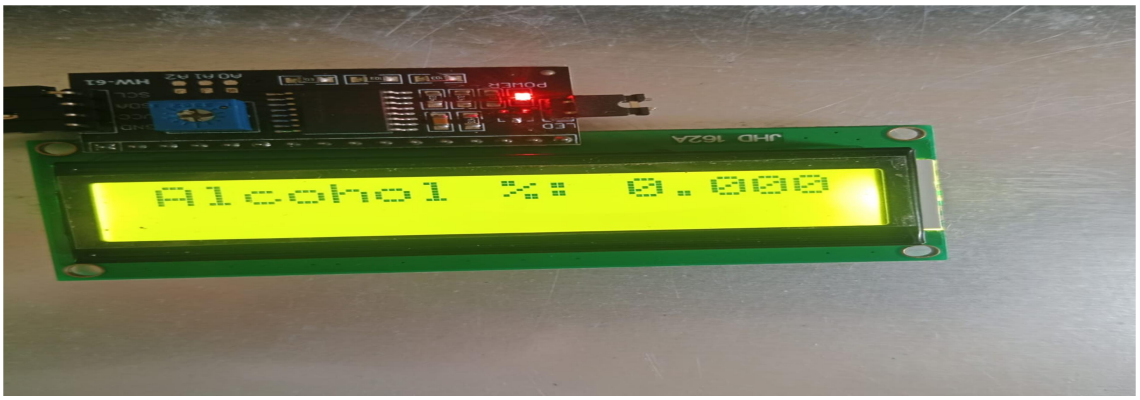


Fig 6.4 Sample Output